

COAL LAB 11 TASK 3

1. Introduction

This program is written in 8086 Assembly Language and executed using EMU8086. It reverses the order of words in a string using stack operations. The stack follows the Last-In-First-Out (LIFO) principle, which helps in reversing data.

2. Objectives

- Understand stack operations (PUSH/POP)
- Learn string traversal in assembly
- Implement word-level reversal logic
- Improve low-level programming skills

3. Flowchart (Step-wise Execution)

START



Load String into SI



Find End of String ('\$')



Move Backward Character by Character



Detect Word (until space)



Push Characters into Stack



Pop Characters from Stack



Print Reversed Word



Repeat for All Words



END

4. Stack Visualization Example

WORD = H E L L O _ W O R L D

Push Order → H, E, L, L, O, (space), W, O, R, L, D

Stack Top → D

Pop Order → D, L, R, O, W, (space), O, L, L, E, H

Output → DLROW OLLEH

5. Program Code with Working Explanation

STEP 1: Starting program

Code:

```
.model small
.stack 100h
.data
str db 'HELLO WORLD$'
.code
main proc
```

Explanation:

- Defines memory structure
- Creates stack
- Stores string
- Starts program execution

STEP 2: Initialize Data Segment

Code:

```
mov ax, @data
mov ds, ax
mov si, offset str
```

Explanation:

- AX loads address of data segment
- DS is initialized so program can access variables
- SI points to start of string "HELLO WORLD"

Now program knows where the string is stored in memory.

STEP 3: Find End of String

Code:

```
find_end:
    mov al, [si]
    cmp al, '$'
    je start_reverse
    inc si
    jmp find_end
```

Explanation:

- Reads characters one by one
- Checks for \$ (end marker)
- If not found → moves forward (SI++)
- Stops when \$ is reached

Now SI is at end of string.

STEP 4: Move One Step Back

Code:

```
start_reverse:
    dec si
```

Explanation:

- Moves back from \$ to last character
- Example:

```
HELLO WORLD$
      ↑ SI was here
now SI → D
```

STEP 5: Start Reverse Loop

Code:

```
reverse_loop:
    cmp si, offset str
    jb done
    mov cx, 0
```

Explanation:

- Checks if SI reached start of string

- If yes → program ends
- CX = 0 → counter for word length

STEP 6: Count Characters of a Word

Code:

```
count_word:
    cmp si, offset str
    jb push_word
    mov al, [si]
    cmp al, ' '
    je push_word
    inc cx
    dec si
    jmp count_word
```

Explanation:

- Moves backward character by character
- Stops when:
 - ❖ space ' ' OR
 - ❖ start of string
- Counts length of word in CX

Example: WORLD → CX = 5

STEP 7: Prepare to Push Word

Code:

```
push_word:
    mov bx, cx
    mov di, si
    inc di
```

Explanation:

- BX = word length
- DI points to first character of word
- inc di moves past space boundary

STEP 8: Push Characters into Stack

Code:

```
push_chars:
    cmp bx, 0
    je print_word
    mov al, [di]
    mov ah, 0
    push ax
    inc di
    dec bx
    jmp push_chars
```

Explanation:

- Push each character into stack
- Stack stores in order:

$W \rightarrow O \rightarrow R \rightarrow L \rightarrow D$

Stack top becomes last character (D)

STEP 9: Pop and Print Word

Code:

```
print_word:
    mov bx, cx

print_chars:
    cmp bx, 0
    je print_space
    pop dx
    mov ah, 2
    int 21h
    dec bx
    jmp print_chars
```

Explanation:

- Pops characters from stack
- Prints them one by one
- Because stack is LIFO → word gets reversed

Output: WORLD

STEP 10: Print Space Between Words

Code:

```
print_space:
    mov dl, ' '
    mov ah, 2
    int 21h
```

Explanation:

- Adds space between reversed words

STEP 11: Move to Next Word

Code:

```
dec si
jmp reverse_loop
```

Explanation:

- Moves left to process previous word
- Repeats entire process

STEP 12: End Program

Code:

```
done:
    mov ah, 4ch
    int 21h
```

Explanation:

- Terminates program safely
- Returns control to DOS

6. Summary of Explanation:

1. DS is initialized
2. SI finds end of string
3. SI moves backward
4. Words are identified
5. Each character is pushed into stack
6. Stack reverses characters
7. Characters are popped and printed
8. Process repeats for all words

7. Output

Input: HELLO WORLD

Output: WORLD HELLO
